Distributed Analytics of Global Torrent Activity

CS404 Big Data Analytics - 1-Week Execution Plan

Shubham Sharma (2023BCS-062) & Ajay Singh (2023BCS-004)

1. Viability Verdict

Viable in one week with a reduced scope. Instead of collecting data slowly over months, the project gets its volume from **public bulk datasets** (billions of rows) and runs a live tracker scraper during the week to demonstrate streaming. Only public metadata is used; no copyrighted content is downloaded and no peer IP addresses are stored.

2. Scope Changes from the Original Abstract

Original plan	1-week version
DHT crawler	Dropped (optional extra via bitmagnet)
Months of hourly tracker scrapes	Scraper runs all week: ~100k torrents x 24h x 6 days = ~14M rows
Decay curves over time	Cross-sectional decay: current seeders vs torrent age (RARBG dates 2007-2023)
Leak lag via IMDb	IMDb has no release dates - use TMDB API (digital release dates) for top ~5-10k titles
Steam pricing + DRM	Kaggle Steam dataset (prices, ratings); DRM via PCGamingWiki only if time allows
Music category	Dropped (little music on public trackers; Spotify API restricted)
Hive, Airflow, Superset	Replaced by Spark SQL; Airflow dropped; Grafana only

3. Datasets (Downloaded Locally)

Dataset	Approx. size	Use
RARBG catalog dump (SQLite)	~2.8M torrents, ~0.5 GB	Core torrent catalog (infohash, title, category, date, IMDb ID)
IMDb datasets (basics, ratings, principals, akas)	100M+ rows, ~1.5 GB	Metadata, genres, ratings, vote counts
Wikipedia hourly pageviews (7-14 days)	1-2+ billion rows, ~8-16 GB gz	Public interest per movie/show/game over time
Kaggle Steam games dataset	~100k games	Prices, ratings, release dates
TMDB API	~5-10k titles	Theatrical / digital release dates
Live tracker scrapes (BEP 15 UDP)	~10-15M rows	Live seeders / leechers - streaming demo

Machine: 81 GB free disk, 15 GB RAM, 12 cores - sufficient for Spark in local mode. Total footprint ~15-25 GB; delete raw .gz files after converting to Parquet. Cloud alternative: BigQuery public Wikipedia pageviews (filter on datehour and wiki to stay in the free 1 TB/month).

4. Big Data Tools and How They Are Used

Tool	Role	Usage in this project
Apache Spark (PySpark)	Processing engine	Reads raw files, cleans, deduplicates, parses titles (guessit), joins all datasets. Runs local[*] on 12 cores.
Spark SQL	Querying	Analytical queries: volume by category/year, leak lag, decay, genre piracy.

Tool	Role	Usage in this project
Apache Parquet	Storage format	Columnar, compressed, partitioned by date/category.
HDFS (optional)	Distributed storage	Single-node Hadoop in Docker holding the Parquet data.
Apache Kafka (optional)	Stream ingestion	Scraper publishes live swarm stats to topic tracker.scrapes.
Spark Structured Streaming	Stream processing	Consumes Kafka, appends to Parquet every minute.
Spark MLlib	Machine learning	Game regression (price/rating/age vs seeders), decay fit, Wikipedia-torrent correlation.
Grafana (or Matplotlib/Plotly)	Visualization	Dashboards for all analyses plus live swarm sizes.
Docker Compose	Environment	One-command, reproducible setup of Kafka, HDFS, Grafana, Postgres.

Must have: PySpark, Spark SQL, Parquet, MLlib, visualization. **Add if time:** Kafka + Structured Streaming (~1 day), then HDFS (~0.5 day).

5. Architecture (Lambda-style: batch + speed + serving layer)

```
Batch layer:
  RARBG dump ---+
  IMDb TSVs  ---+
  Wikipedia  ---+--> PySpark ETL --> Parquet (HDFS / local)
  Steam/TMDB ---+                          |
                                           +--> Spark SQL analyses --+
                                           +--> Spark MLlib models --+--> Grafana
Speed layer:                               |
  Tracker scraper --> Kafka --> Spark Structured Streaming --+  (same Parquet store)
```

6. Analyses

- Torrent volume by category and year (RARBG, 2007-2023) - cross-media activity index
- Popularity decay: seeders vs torrent age, exponential / power-law fit
- Leak lag: TMDB digital release date vs first torrent upload
- Public interest vs piracy: Wikipedia pageview spikes vs torrent swarm size, per category
- Games: price and rating vs torrent activity (MLlib regression)
- Genres most pirated relative to popularity (IMDb vote-count join)

All results are correlations / associations, not causal effects.

7. Division of Work

Person A - Data Engineering (Shubham)	Person B - Analytics (Ajay)
Docker Compose (Kafka, HDFS, Grafana) Download scripts Tracker scraper -> Kafka -> Structured Streaming PySpark ETL to clean Parquet, title parsing, dedup	TMDB fetcher Spark SQL analyses Spark MLlib models Grafana dashboards Most of report and slides

Day	Person A	Person B
1	Start all downloads and the tracker scraper (JSONL for now)	Agree Parquet schema contract with A; get TMDB key, start fetching release dates
2	Docker: Kafka + HDFS up; Spark streaming job Kafka -> Parquet	Write SQL analyses against sample data (first 100k rows)
3	Full ETL of RARBG, IMDb, Wikipedia, Steam to Parquet; guessit parsing	Finish analyses 1-4 on sample; start MLlib regression
4	Joins (torrents + IMDb + TMDB + Wikipedia + Steam); hand over final tables	Run all analyses on full data; finish MLlib models

Day	Person A	Person B
5	Performance tuning; runtime / data-volume stats for report	Grafana dashboards
6	Report: architecture, pipeline, tools	Report: results, charts, limitations; slides
7	Together: full demo rehearsal, bug fixes, final screenshots	

8. Rules and Risks

- **Start the scraper on Day 1** - every day it does not run is data lost for good.
- Agree the schema on Day 1, e.g. torrents(infohash, title, category, year, imdb_id, size, added_at), so B is never blocked by A.
- One shared Git repo: ingest/ and etl/ (A), analysis/ and dashboards/ (B), report/ (both).
- 15-minute check-in every evening: done / blocked / tomorrow.
- Feature freeze at end of Day 5; after that only fixes and writing.
- One machine with Docker Compose - no multi-VM cluster. Use Bitnami images if setup stalls.
- Both members must be able to explain the whole pipeline - examiners ask about the other half.
- Update the submitted abstract to match the tools actually used.